

Labs 8

This week covers topics from

- Distance Metrics
- Clustering
- Silhouettes

Submission And Assignment Instructions

Submit your `Lab8_lastname_firstname.pdf` file on canvas with your answers clearly marked (✓) and code commented. Skeleton code and/or comments are provided throughout to help you.

Formatting Notes:

- ? Questions you must answer or tasks are marked with a " ? "
- ✓ Answers you should give are marked with "✓ Answer:"
- *Helpful hints are usually given in italics*
- Code you need to write is marked with `## ? YOUR CODE HERE`
 - ★ Bonus questions are marked with a "★ "
 - 🍓 Bonus answers are marked with a "🍓 "

Please don't erase ANY of these markings! They're to help the TAs find your answers and code as much as they're to help you find the questions.

Section 1: Proof of Non-negativity

? Prove that non-negativity ($0 \leq D(\vec{x}, \vec{y})$) follows from the three properties of a similarity metric. *You may do this math on paper and insert a picture of your work here.*

```
In [1]: from IPython.display import Image, display
display(Image("1.jpg"))
```

Metrics follow 3 main properties:

$$1) \text{ Identity : } D(x, x) = 0$$

$$2) \text{ Symmetry : } D(x, y) = D(y, x)$$

$$3) \text{ Triangle inequality : } D(x, z) \leq D(x, y) + D(y, z)$$

Proof for $(0 \leq D(\vec{x}, \vec{y}))$:

set $z = x$ in the triangle inequality

$$D(x, x) \leq D(x, y) + D(y, x)$$

using identity +
symmetry property

$$0 \leq D(x, y) + D(x, y)$$

$$0 \leq 2D(x, y)$$

$$0 \leq D(x, y)$$

Section 2 - Real data

We've seen clusters or groups of data many times in class, usually clusters from externally generated labels. Now we're going to use unsupervised clustering to explore the structure of our data.

2.1: Load the dataset

This dataset is from a [paper](#) investigating subtypes of T-lineage acute lymphoblastic leukemia.

? Load the truncated provided datafile (`data_09_TALL_tpm_reduced.csv`).

Some checks to help you:

- This dataset is TPM (transcripts per million) gene expression data from 1,335 different tumor samples across 300 genes. Check that your dataset is the expected size after loading it.
- The first column of your data should be long strings (e.g. `ENSG000000004939.15`). These are ENSEMBL IDs, identification strings for particular gene isoforms. *Load these as the index column of your dataframe.*

```
In [2]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# load local file "data08.csv"
#from google.colab import files
'''
uploaded = files.upload()

for fn in uploaded.keys():
    print('User uploaded file "{name}" with length {length} bytes'.format(
        name=fn, length=len(uploaded[fn])))
'''

## ? YOUR CODE HERE
# load dataset
df = pd.read_csv("data_09_TALL_tpm_reduced.csv", index_col=0)

# check dataset
print(df.shape)
df.head()
```

(300, 1335)

Out [2]:

	PAUCKE	PARLHD	PARMJA	PARVRF	PARYMD	PASXLM	PAT
--	--------	--------	--------	--------	--------	--------	-----

X							
ENSG00000000971.15	0.03	0.11	0.01	4.73	6.61	0.12	
ENSG00000001626.15	0.00	0.00	0.20	0.00	0.16	0.00	
ENSG00000003436.16	3.77	3.75	9.16	2.89	11.02	23.81	
ENSG00000004939.15	0.39	6.40	31.64	2.24	45.40	0.12	
ENSG00000005381.7	0.30	6.47	1.22	0.81	0.73	7.81	

5 rows × 1335 columns

? Once again, we've loaded gene expression data with genes as rows and samples as columns. Transpose the data to make the genes (features) into columns and the column names into sample IDs.

```
In [4]: # YOUR CODE HERE
df_trans = df.T
```

```
print(df_trans.shape)
```

```
(1335, 300)
```

2.2: Hierarchical Clustering

We're going to use hierarchical clustering to visualize these data and create clusters. Since this is a large dataset, we're going to compute the linkage matrix separately using `fastcluster` before visualizing with seaborn.

? 2.2 Use `fastcluster.linkage` to compute the hierarchical clustering with complete linkage. This output is called a stepwise dendrogram. Print the first several (look at 15-20) rows of the stepwise dendrogram numpy array.

```
In [5]: # %%capture
!pip install fastcluster
import fastcluster
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster

## YOUR CODE HERE
output = fastcluster.linkage(df_trans, method='complete')

print(output[:20])
```

Collecting fastcluster

Downloading fastcluster-1.3.0-cp311-cp311-macosx_11_0_arm64.whl.metadata (4.7 kB)

Requirement already satisfied: numpy>=2 in /Users/dianadayekh/anaconda3/envs/clean_cluster/lib/python3.11/site-packages (from fastcluster) (2.4.2)

Downloading fastcluster-1.3.0-cp311-cp311-macosx_11_0_arm64.whl (34 kB)

Installing collected packages: fastcluster

Successfully installed fastcluster-1.3.0

```
[ [ 335.          925.          49.86833163    2.          ]
  [   3.          1335.         70.12810635    3.          ]
  [ 900.          976.          86.18298556    2.          ]
  [ 350.          849.          92.43486518    2.          ]
  [  66.          267.          93.87894865    2.          ]
  [1015.         1043.          97.99988418    2.          ]
  [ 269.         1336.         104.17701474    4.          ]
  [ 456.         1337.         110.70595241    3.          ]
  [ 461.          893.         112.19822191    2.          ]
  [1032.         1343.         131.14502316    3.          ]
  [ 399.          938.         137.59028381    2.          ]
  [1342.         1344.         141.556766     6.          ]
  [  30.           47.         142.39807197    2.          ]
  [ 626.         1262.         142.594313     2.          ]
  [ 553.          714.         143.38061619    2.          ]
  [ 658.          685.         143.83076514    2.          ]
  [ 487.         1339.         144.50486843    3.          ]
  [ 964.         1266.         149.89764508    2.          ]
  [ 660.         1123.         151.25511925    2.          ]
  [1248.         1345.         151.98152256    3.          ]]
```

A couple questions about this output:

- ? 2.2.1 What does each row of this output represent? (1 sentence)
 - ✓ Answer: Each row represents one merge step in the hierarchical clustering process.
- ? 2.2.2 What does each column of this output represent? *There should be 4 columns.*
 - ✓ Answer: Column 1 represents the first cluster being merged, column 2 represents the second cluster being merged, column 3 represents the distance between the two clusters at the time of merging (using complete linkage, this is the maximum distance between any two points in the two clusters), and column 4 represents the number of samples contained in the newly formed cluster after the merge.
- ? 2.2.3 How many rows does this output have? How are the rows of data arranged? (1 sentence)
 - ✓ Answer: To go from 1335 clusters to 1 cluster, we must perform 1334 merges. Therefore there are 1334 rows (given that each row represents one merge). The rows are arranged in the order the merges happen, from the smallest distance to the largest distance.
- ? 2.2.4 By this linkage metric, what are the indices of the two most similar samples? *These are tumor sample indices, not gene indices, since we're computing the clustering on rows of our dataframe. One way to know that is that you have only 300 genes, and both these indices are >300!*
 - ✓ Answer: The two most similar tumor samples are 335 and 925.
- ? 2.2.5 The first cluster formed has index 1335. At what distance does this cluster recombine with another cluster for the second time? *Hint: You should not have to print out more rows of this output or write more code to answer this question.*
 - ✓ Answer: Cluster 1335 recombines with another cluster (cluster 3) at a distance of 70.13 (shown by row 2)

2.3: Visualizing Clustering

? 2.3 Use `sns.clustermap` to visualize these data into a complete linkage heatmap. *This question has many steps, please read carefully.*

Make sure to:

- use the `fcluster` function to divide your clustered rows (samples) into 15 clusters

- `fcluster` lets you divide a dendrogram into clusters multiple ways. Rather than choosing a height on the dendrogram, choose a maximum number of clusters.
- use the `row.colors` argument in `sns.clustermap` to visually divide the rows (samples) into those 15 distinct clusters.
- provide the row linkage clustering you already computed using the `row_linkage` argument of the `sns.clustermap` function.
- include a title to identify the type of clustering you used and other relevant information (clustering type, linkage type, distance metric)
- label your colorbar (what is this scale?)
- use `standard_scale = 1` to map the expression of each gene onto a scale between 0 and 1 (hint: this changes what the colorbar scale is). *Does this change how the row linkage is produced?*
- use `method = "complete"` to use complete linkage clustering on the genes/columns (in addition to the rows, which you have already clustered).
- label the x and y-axes as either "Genes" or "Samples", but remove individual sample or ENSEMBL IDs for brevity
- include the clustering dendrograms for both genes and samples in your final heatmap

```
In [6]: # %%capture
# !pip install fastcluster
import fastcluster
from scipy.cluster.hierarchy import dendrogram, fcluster

## ? YOUR CODE HERE

# cut the dendrogram to create clusters
num_clusters = 15
row_clusters = fcluster(output, t=num_clusters, criterion='maxclust')

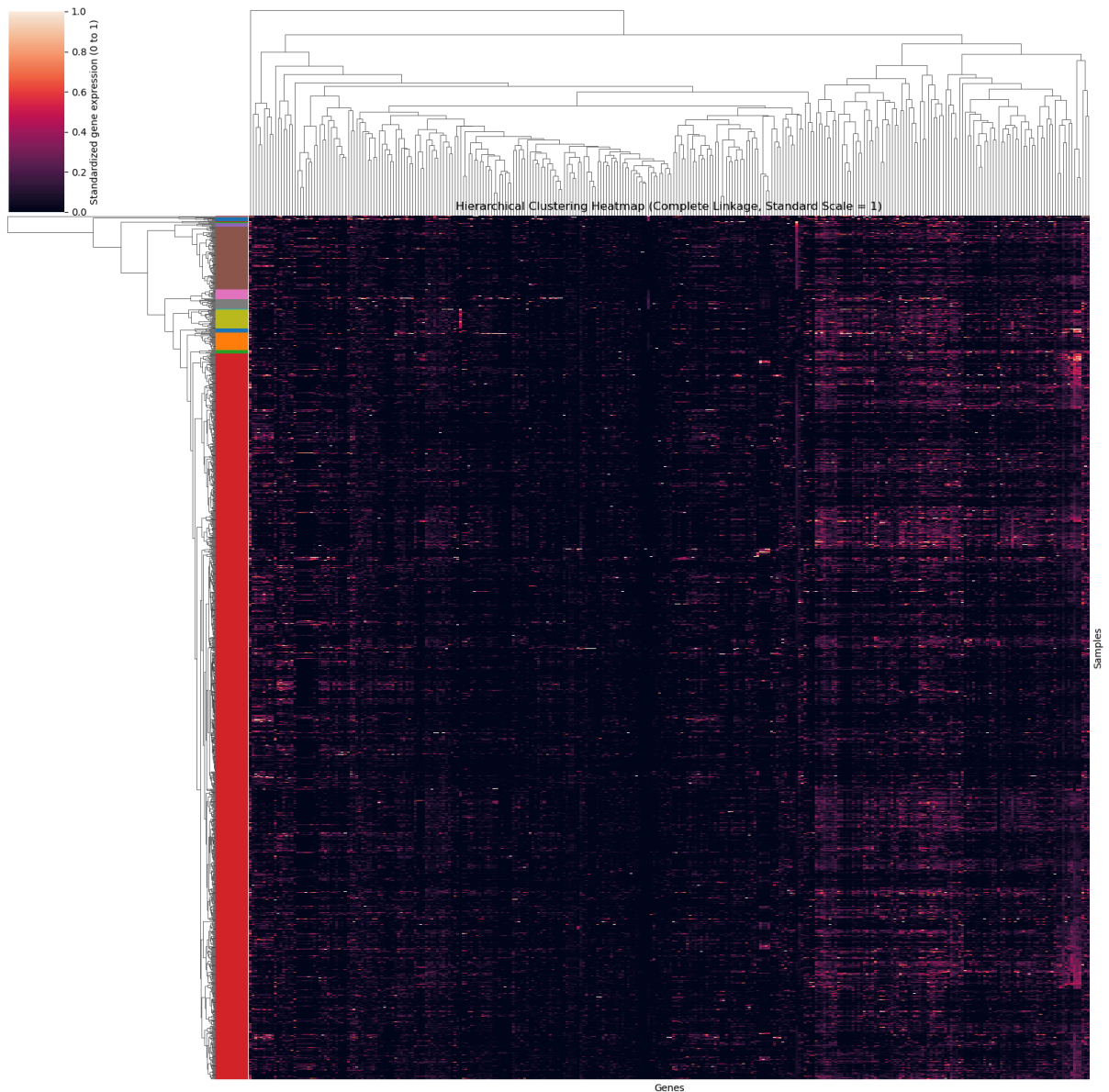
# Example: Create a color palette for the clusters
palette = sns.color_palette("tab10", num_clusters)
cluster_colors = [palette[i-1] for i in row_clusters]

# Create a clustermap with colored row labels based on clusters
# make sure to label the colorbar
g = sns.clustermap(
    df_trans,
    row_linkage=output,
    row_colors=cluster_colors,
    method="complete",
    standard_scale=1,
    xticklabels=False,
    yticklabels=False,
    figsize=(16,16),
    cbar_kws={"label": "Standardized gene expression (0 to 1)"}
)

# add axis labels and title
```

```
g.ax_heatmap.set_xlabel("Genes")
g.ax_heatmap.set_ylabel("Samples")
g.ax_heatmap.set_title("Hierarchical Clustering Heatmap (Complete Linkage, S
```

Out[6]: Text(0.5, 1.0, 'Hierarchical Clustering Heatmap (Complete Linkage, Standard Scale = 1)')



2.4 Comparing Types of Clustering

? 2.4.1 Compare single-linkage, complete-linkage, and the average linkage on this data, plotting each heatmap as before (you should need 2 additional heatmaps). Which sort of linkage gives the most uneven clusters by size (number of samples per cluster)? Based on what you know about how that linkage is calculated, why is that the case?

```
In [7]: ## YOUR CODE HERE
        #Single Linkage

        row_link_single = fastcluster.linkage(df_trans, method="single")
```



```

row_clusters_single = fcluster(row_link_single, t=15, criterion='maxclust')

palette = sns.color_palette("husl", 15)
cluster_colors_single = [palette[i-1] for i in row_clusters_single]

single_linkage = sns.clustermap(
    df_trans,
    row_linkage=row_link_single,
    row_colors=cluster_colors_single,
    method="single",
    standard_scale=1,
    xticklabels=False,
    yticklabels=False,
    figsize=(16,16),
    cbar_kws={"label": "Standardized gene expression (0 to 1)"}
)

single_linkage.ax_heatmap.set_xlabel("Genes")
single_linkage.ax_heatmap.set_ylabel("Samples")
single_linkage.fig.suptitle("Hierarchical Clustering Heatmap (Single Linkage)")

#Average Linkage
row_link_avg = fastcluster.linkage(df_trans, method="average")
row_clusters_avg = fcluster(row_link_avg, t=15, criterion='maxclust')

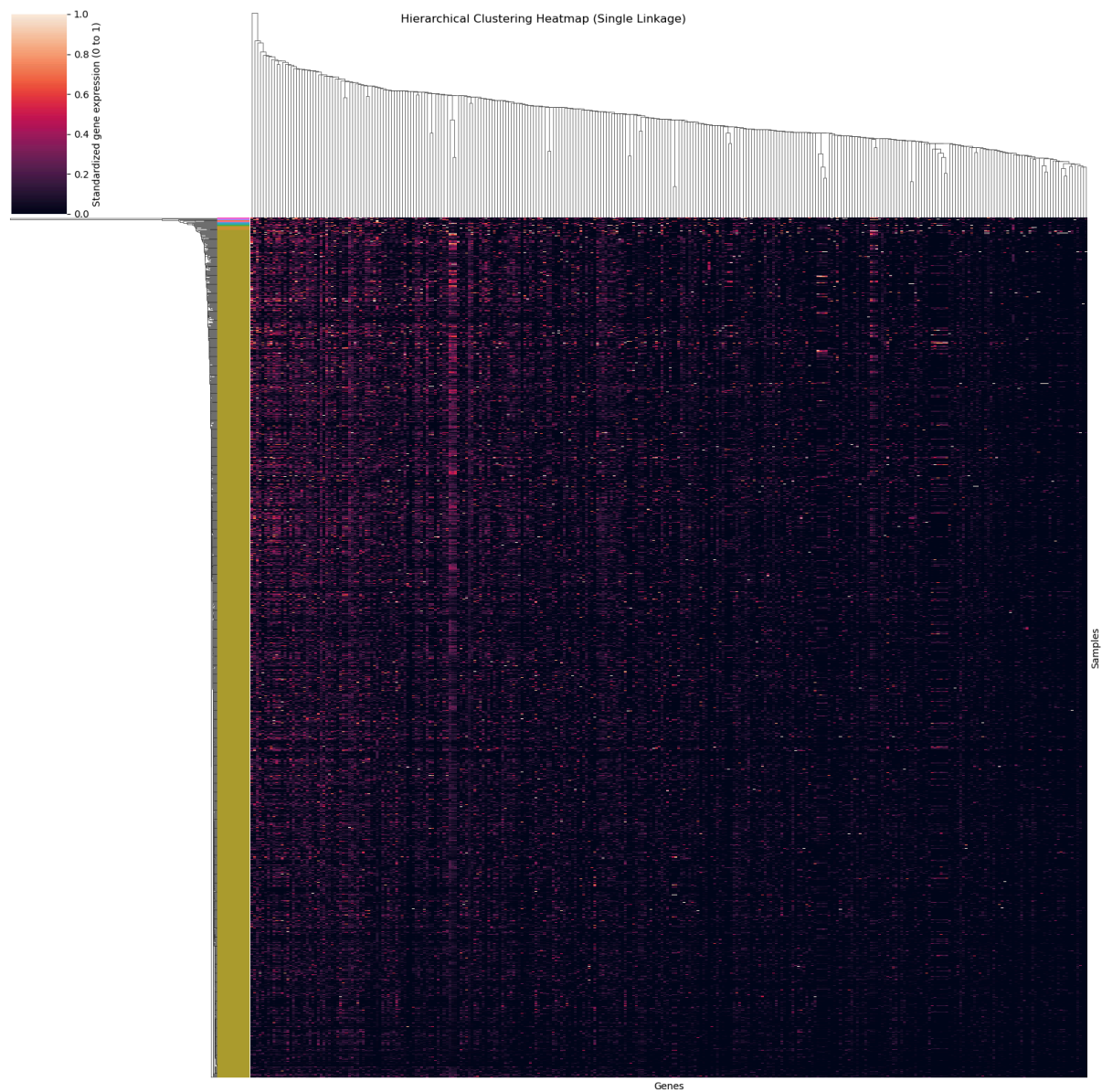
palette = sns.color_palette("husl", 15)
cluster_colors_avg = [palette[i-1] for i in row_clusters_avg]

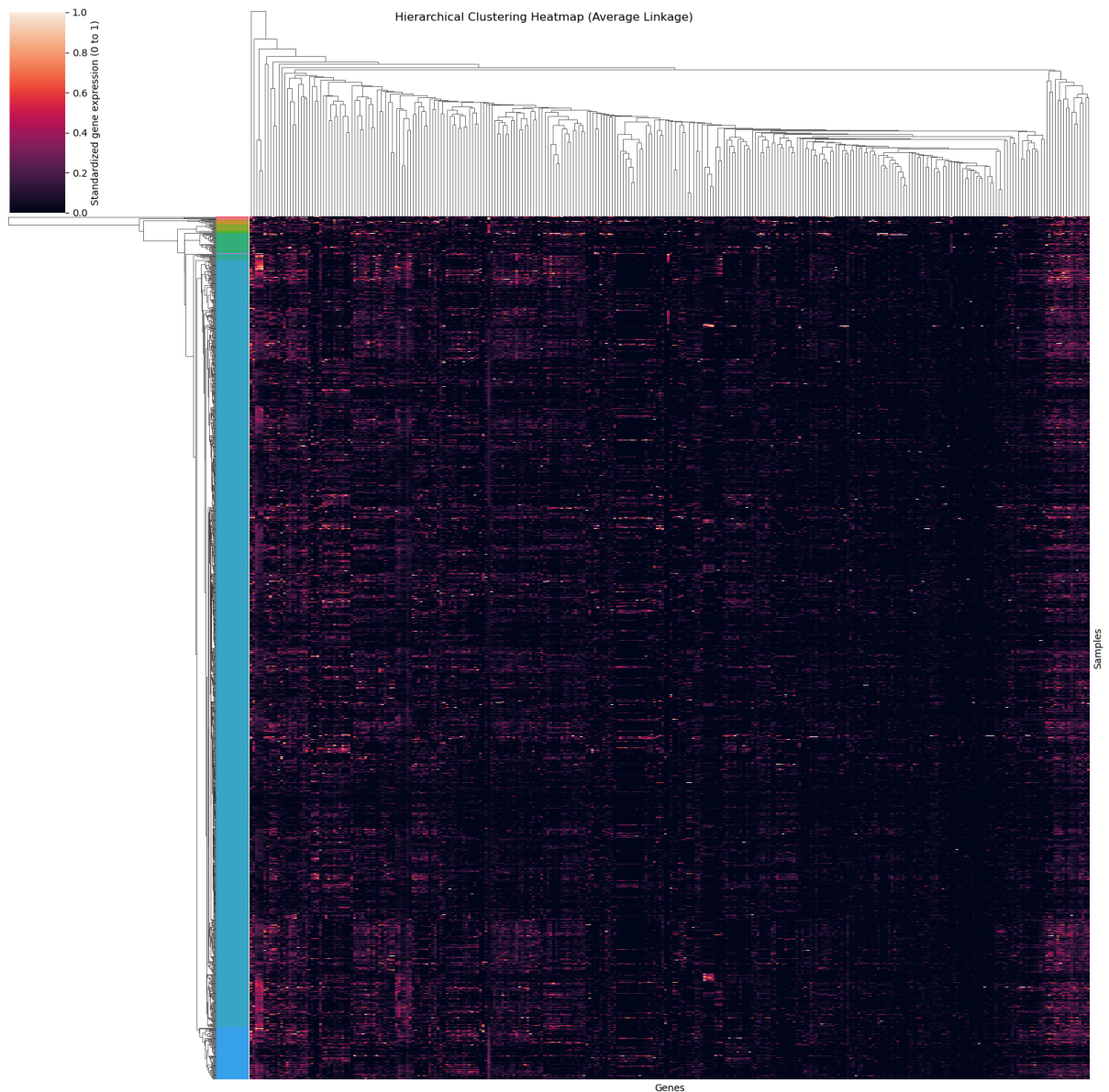
average_linkage = sns.clustermap(
    df_trans,
    row_linkage=row_link_avg,
    row_colors=cluster_colors_avg,
    method="average",
    standard_scale=1,
    xticklabels=False,
    yticklabels=False,
    figsize=(16,16),
    cbar_kws={"label": "Standardized gene expression (0 to 1)"}
)

average_linkage.ax_heatmap.set_xlabel("Genes")
average_linkage.ax_heatmap.set_ylabel("Samples")
average_linkage.fig.suptitle("Hierarchical Clustering Heatmap (Average Linkage)")

```

Out[7]: Text(0.5, 0.98, 'Hierarchical Clustering Heatmap (Average Linkage)')





✓ Answer:

Which sort of linkage gives the most uneven clusters by size (number of samples per cluster)? Single linkage gives the most uneven clusters by size. In the single-linkage heatmap, the colored bar on the left shows that one color dominates a very large portion of the samples, while only a few samples belong to other colors. This means one cluster contains many samples while the other clusters are much smaller, making the cluster sizes very uneven.

Based on what you know about how that linkage is calculated, why is that the case? Single linkage calculates the distance between clusters using the minimum distance between any two points in the clusters. Because of this, clusters can merge as long as one pair of samples is close together, even if the rest of the samples are not very similar. This creates a chaining effect, where samples keep linking to an existing cluster, eventually forming one very large cluster while leaving several much smaller clusters.

? 2.4.2 Are these hierarchical clustering methods sensitive to the scale of the data?

Write some code to illustrate your answer.

```
In [8]: ## YOUR CODE HERE
df_changed = df_trans.copy()
df_changed.iloc[:, 0] = df_changed.iloc[:, 0] * 1000 # one gene is scaled

linkage_changed = fastcluster.linkage(df_changed, method="complete")
clusters_changed = fcluster(linkage_changed, t=15, criterion="maxclust")

print("Original clusters:")
print(row_clusters[:20])

print("Clusters after scaling one gene column:")
print(clusters_changed[:20])

#prints out the cluster assignment of the first 20 samples
```

Original clusters:

```
[14 14 14 14 14 14 14 14 14 14 14 14 14 14 14 14 6 14 14 14]
```

Clusters after scaling one gene column:

```
[11 11 11 13 14 11 13 11 11 13 14 11 11 11 11 13 8 13 11 13]
```

✓ Answer (1 sentence): Yes, hierarchical clustering methods are sensitive to the scale of the data because they rely on distance calculations. Changing the scale of the features changes the distances between samples, which can change how clusters are formed.

? 2.4.3 For each of these hierarchical methods, given the data that you have visualized, do you think choosing $n = 15$ clusters makes the most sense, based on the dendrograms? Why or why not?

✓ Answer: No, $n=15$ does not seem like the most natural choice for all three methods. The dendrograms do not show a clear split into exactly 15 groups. In particular, single linkage shows a chaining effect with one very large cluster and several very small ones, so 15 clusters is not a very meaningful cut. Average and complete linkage look more reasonable, but even there the dendrograms do not strongly suggest exactly 15 clusters.

2.5 Splits

? We didn't use a train-test split to try to evaluate any of these clustering techniques! Should we have used a train-test split? Why or why not? *Hint: Were these supervised or unsupervised techniques?*

✓ Answer(1 sentence): Hierarchical clustering is unsupervised and does not rely on labeled data. Since there are no true labels to evaluate predictions against, a train-test split is not required.

Section 3 - Quantitative Comparison on Toy data

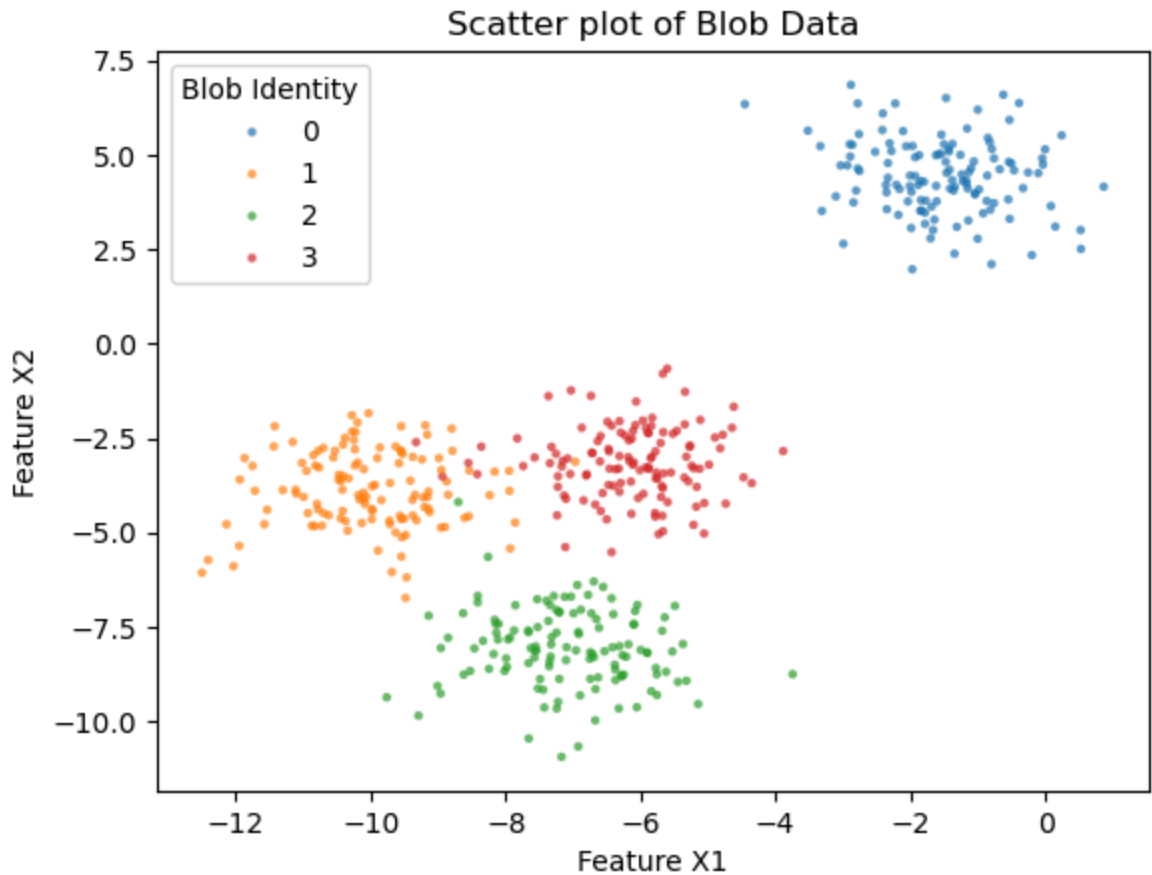
We've been arbitrarily dividing our tumor samples into 15 clusters, which doesn't seem to be a good fit for the dendrograms. We're going to take a quick break from the real data to use some toy data to understand k-means clustering and some quantitative measures that can help us evaluate different cluster formations.

? 3.1 Run the code below to create and visualize 4 clusters of toy data. Write additional code to do hierarchical clustering on these data using an appropriate linkage type and print a heatmap. As before, title and label your heatmap and accompanying colorbar. Also mark the true cluster identity `row_colors` on the heatmap.

```
In [9]: from sklearn.datasets import make_blobs

# generate toy data
X, y = make_blobs(
    n_samples=500,
    n_features=2,
    centers=4,
    cluster_std=1,
    center_box=(-10.0, 10.0),
    shuffle=True,
    random_state=1)
X[0:5]
# This is only 2-D data, so we can directly visualize it!
# plotting
data = pd.DataFrame(X, columns=["X1", "X2"])
data['cluster'] = y
sns.scatterplot(data, x = "X1", y = "X2", hue = "cluster", marker=".",
                s=40, lw=0, alpha=0.7, palette='tab10')
plt.xlabel("Feature X1")
plt.ylabel("Feature X2")
plt.title("Scatter plot of Blob Data")
plt.legend(title="Blob Identity", loc="upper left")
```

```
Out[9]: <matplotlib.legend.Legend at 0x16dc0f390>
```



```
In [11]: ## YOUR CODE BELOW

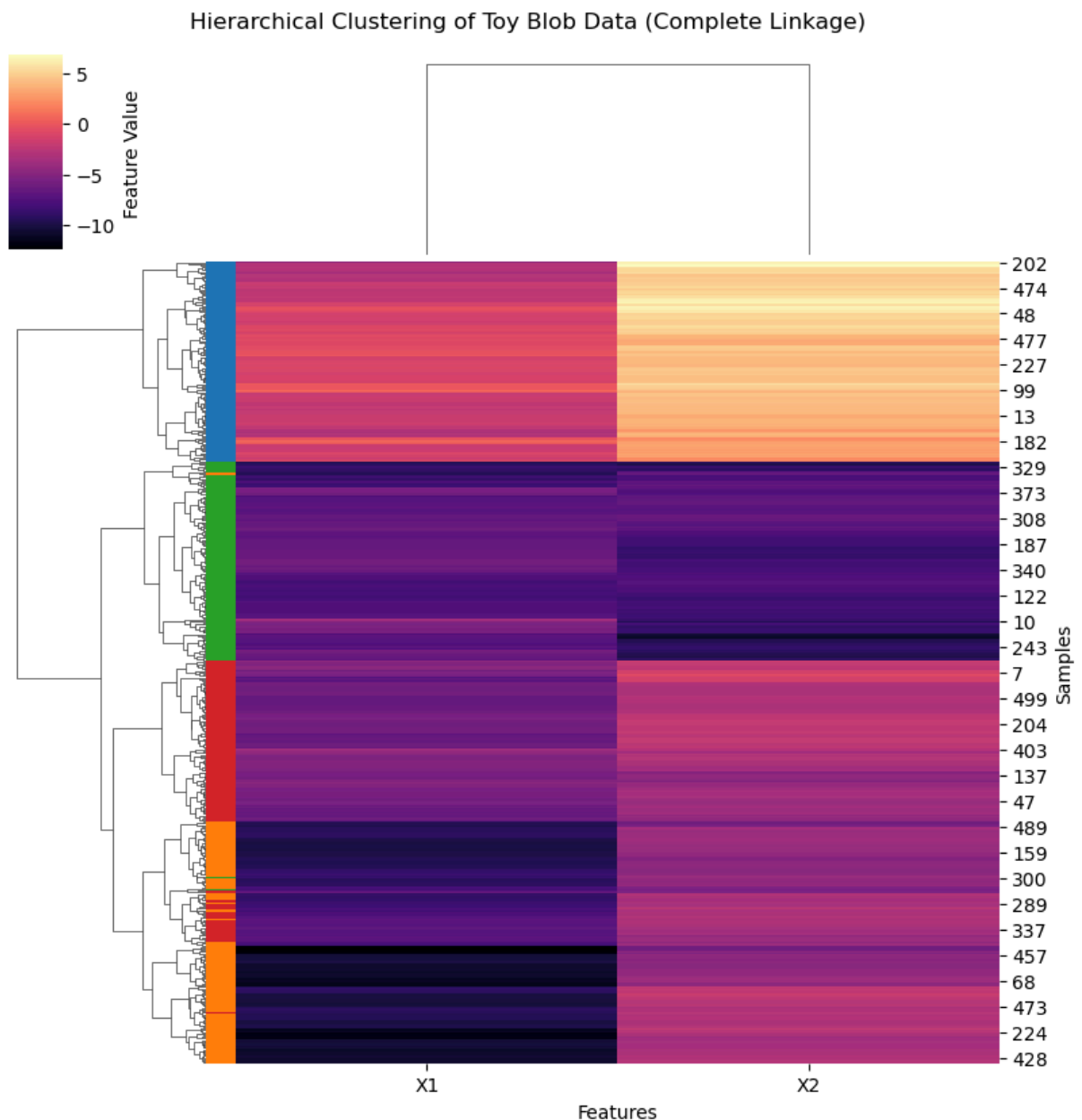
# create hierarchical clustering heatmap with 2 features
# color by true blob identity

palette = sns.color_palette("tab10", 4)
row_colors = [palette[i] for i in y]
row_linkage = fastcluster.linkage(data[["X1","X2"]], method="complete")

g = sns.clustermap(
    data[["X1","X2"]],
    row_linkage=row_linkage,
    row_colors=row_colors,
    method="complete",
    cmap="magma",
    figsize=(8,8),
    cbar_kws={"label": "Feature Value"}
)

g.ax_heatmap.set_xlabel("Features")
g.ax_heatmap.set_ylabel("Samples")
g.fig.suptitle("Hierarchical Clustering of Toy Blob Data (Complete Linkage)")

Out[11]: Text(0.5, 1.02, 'Hierarchical Clustering of Toy Blob Data (Complete Linkage)')
```



? 3.2 Cut the dendrogram at using a distance threshold and `fcluster`. Play with the distance threshold until you get 4 clusters. Create 3 plots:

1. A heatmap (as above) of the clustered features with the `row_colors` label reflecting the clusters chosen by `fcluster`.
2. A silhouette plot for these chosen clusters (hint: you might want to look ahead at the provided code for the next problem).
3. A scatterplot of the data, colored by the chosen cluster identity

```
In [18]: from sklearn.metrics import silhouette_samples, silhouette_score
from sklearn.cluster import KMeans
import matplotlib.cm as cm

## YOUR CODE HERE

# cut the dendrogram using a distance threshold = choose a height on the der
```



```

distance_threshold = 7
cluster_labels = fcluster(row_linkage, t=distance_threshold, criterion="dist")
print("Number of clusters:", len(np.unique(cluster_labels)))

# create row colors for the heatmap
palette = sns.color_palette("tab10", len(np.unique(cluster_labels)))
row_colors_2 = [palette[i-1] for i in cluster_labels]

# 1. heatmap with row colors showing chosen clusters
g = sns.clustermap(
    data[["X1", "X2"]],
    row_linkage=row_linkage,
    row_colors=row_colors_2,
    method="complete",
    cmap="magma",
    figsize=(8, 8),
    cbar_kws={"label": "Feature Value"}
)

g.ax_heatmap.set_xlabel("Features")
g.ax_heatmap.set_ylabel("Samples")
g.fig.suptitle("Hierarchical Clustering Heatmap (Distance Cut)", y=1.02)
plt.show()

# 2. silhouette plot
silhouette_vals = silhouette_samples(data[["X1", "X2"]], cluster_labels)
silhouette_avg = silhouette_score(data[["X1", "X2"]], cluster_labels)

fig, ax = plt.subplots(figsize=(6, 4))
y_lower = 10

for i in np.unique(cluster_labels):
    cluster_sil_vals = silhouette_vals[cluster_labels == i]
    cluster_sil_vals.sort()
    size_cluster_i = len(cluster_sil_vals)
    y_upper = y_lower + size_cluster_i

    ax.fill_betweenx(
        np.arange(y_lower, y_upper),
        0,
        cluster_sil_vals,
        alpha=0.7
    )

    ax.text(-0.05, y_lower + 0.5 * size_cluster_i, str(i))
    y_lower = y_upper + 10

ax.axvline(x=silhouette_avg, color="red", linestyle="--")
ax.set_xlabel("Silhouette coefficient")
ax.set_ylabel("Bar Position")
ax.set_title("Silhouette Plot for Hierarchical Clustering")
plt.show()

# 3. scatterplot colored by chosen cluster identity
X, y = make_blobs(

```



```

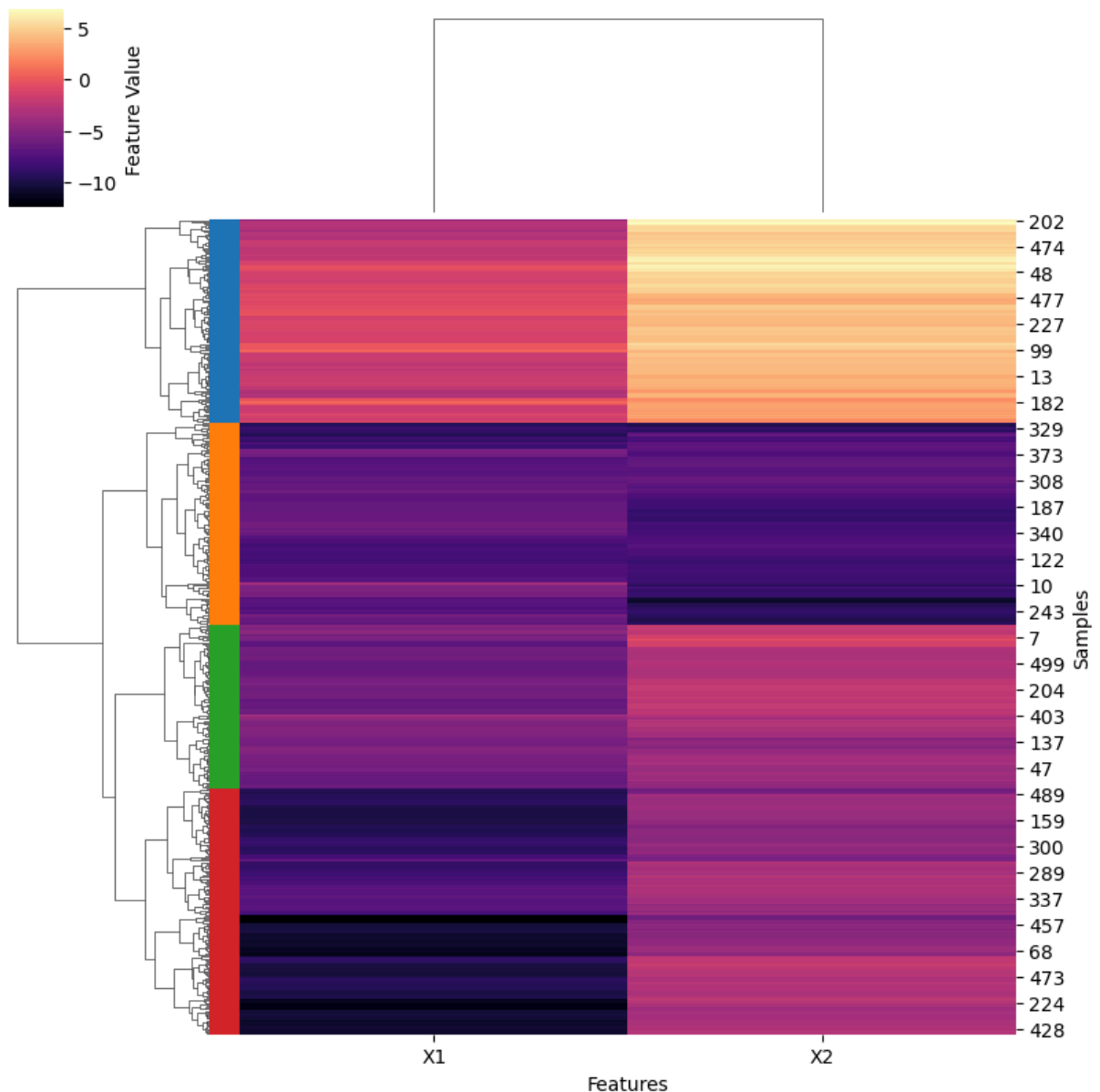
n_samples=500,
n_features=2,
centers=4,
cluster_std=1,
center_box=(-10.0, 10.0),
shuffle=True,
random_state=1)
X[0:5]

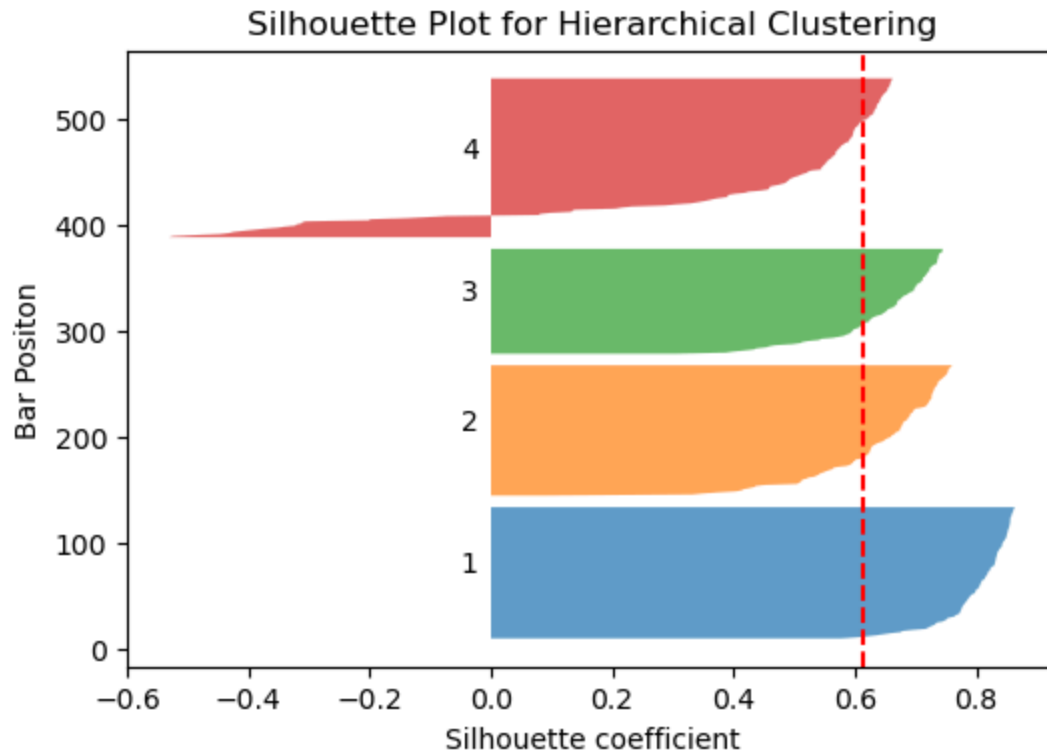
data = pd.DataFrame(X, columns=["X1", "X2"])
data['cluster'] = y
sns.scatterplot(data, x = "X1", y = "X2", hue = "cluster", marker=".",
                s=40, lw=0, alpha=0.7, palette='tab10')
plt.xlabel("Feature X1")
plt.ylabel("Feature X2")
plt.title("Scatter plot of Blob Data")
plt.legend(title="Blob Identity", loc="upper left")

```

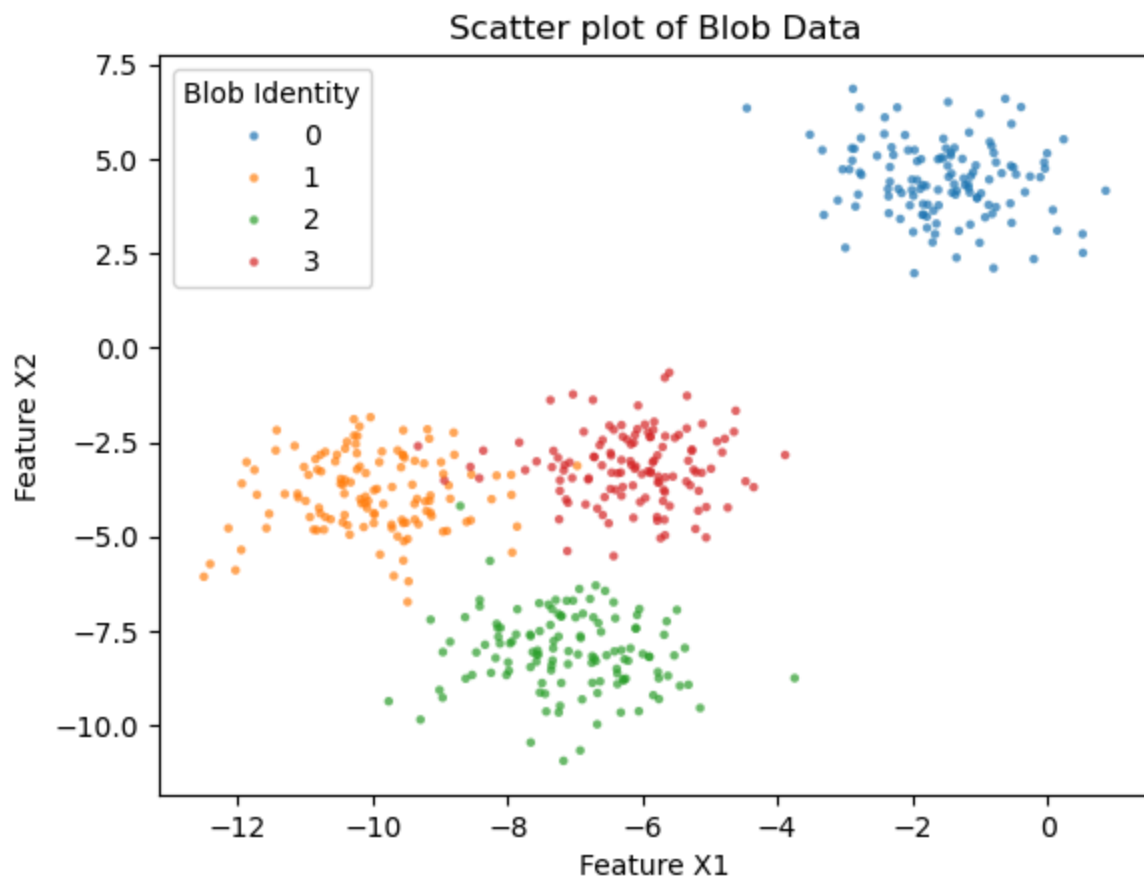
Number of clusters: 4

Hierarchical Clustering Heatmap (Distance Cut)





Out[18]: <matplotlib.legend.Legend at 0x149c2bf10>



? 3.3 Run the code below to cluster these data using k-means with $k=2, 3, 4, 5$, and 6 (use a for loop!). Inspect the resulting plots. Which value of k best suits these data? Is

that the value you'd expect? Using what you know about silhouette scores, why is this the case?

```
In [19]: import matplotlib.cm as cm

range_n_clusters = [2, 3, 4, 5, 6]

for n in range_n_clusters:
    fig, (ax1, ax2) = plt.subplots(1, 2)
    fig.set_size_inches(18, 7)
    ax1.set_xlim([-0.1, 1])
    ax1.set_ylim([0, len(X) + (n + 1) * 10])

    cluster = KMeans(n_clusters=n, random_state=10)
    cluster_labels = cluster.fit_predict(X)
    silhouette_avg = silhouette_score(X, cluster_labels)
    print(f'For n_clusters={n}, average silhouette score is: {silhouette_avg}')
    sample_silhouette_values = silhouette_samples(X, cluster_labels)

    y_lower = 10
    for i in range(n):
        ith_cluster_silhouette_values = sample_silhouette_values[cluster_labels == i]
        ith_cluster_silhouette_values.sort()
        size_cluster_i = ith_cluster_silhouette_values.shape[0]
        y_upper = y_lower + size_cluster_i

        color = cm.nipy_spectral(float(i) / n)
        ax1.fill_betweenx(
            np.arange(y_lower, y_upper),
            0,
            ith_cluster_silhouette_values,
            facecolor=color,
            edgecolor=color,
            alpha=0.7,
        )
        ax1.text(-0.05, y_lower + 0.5 * size_cluster_i, str(i))
        y_lower = y_upper + 10

    ax1.set_title("The silhouette plot for the various clusters.")
    ax1.set_xlabel("The silhouette coefficient values")
    ax1.set_ylabel("Cluster label")
    ax1.axvline(x=silhouette_avg, color="red", linestyle="--")
    ax1.set_yticks([])
    ax1.set_xticks([-0.1, 0, 0.2, 0.4, 0.6, 0.8, 1])
    colors = cm.nipy_spectral(cluster_labels.astype(float) / n)
    ax2.scatter(
        X[:, 0], X[:, 1], marker=".", s=30, lw=0, alpha=0.7, c=colors, edgecolor='k'
    )

    centers = cluster.cluster_centers_
    ax2.scatter(
        centers[:, 0],
        centers[:, 1],
        marker="o",
```

```

        c="white",
        alpha=1,
        s=200,
        edgecolor="k",
    )

    for i, c in enumerate(centers):
        ax2.scatter(c[0], c[1], marker="$%d$" % i, alpha=1, s=50, edgecolor=

    ax2.set_title("The visualization of the clustered data.")
    ax2.set_xlabel("Feature space for the 1st feature")
    ax2.set_ylabel("Feature space for the 2nd feature")

    plt.suptitle(
        "Silhouette analysis for KMeans clustering on sample data with n_clu
        % n,
        fontsize=14,
        fontweight="bold",
    )

plt.show()

```

For n_clusters=2, average silhouette score is: 0.7049787496083262

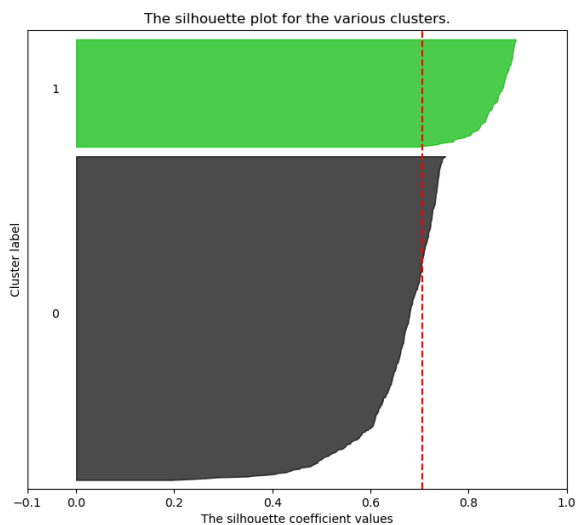
For n_clusters=3, average silhouette score is: 0.5882004012129721

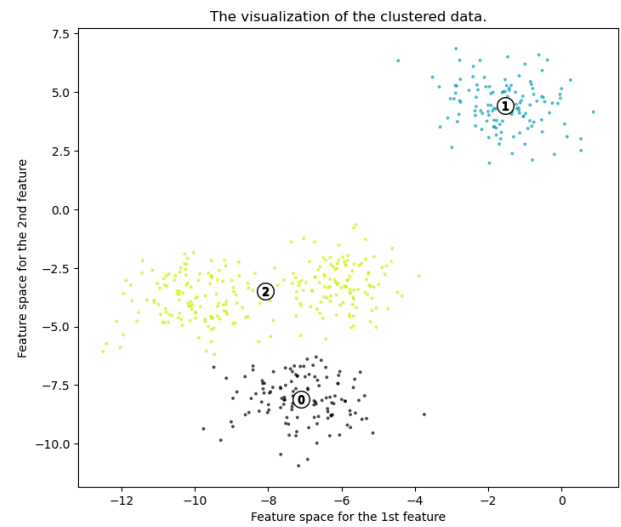
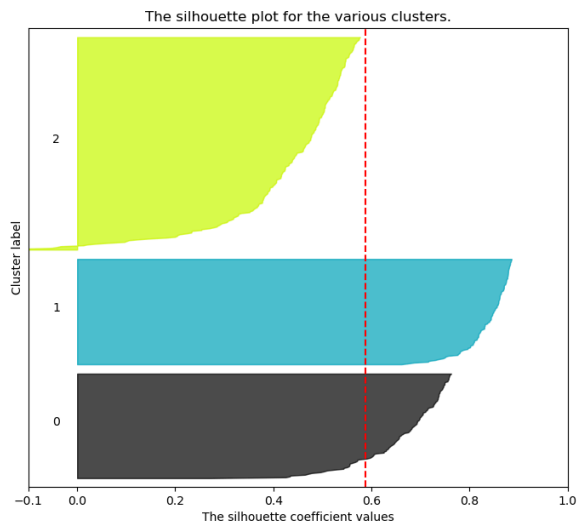
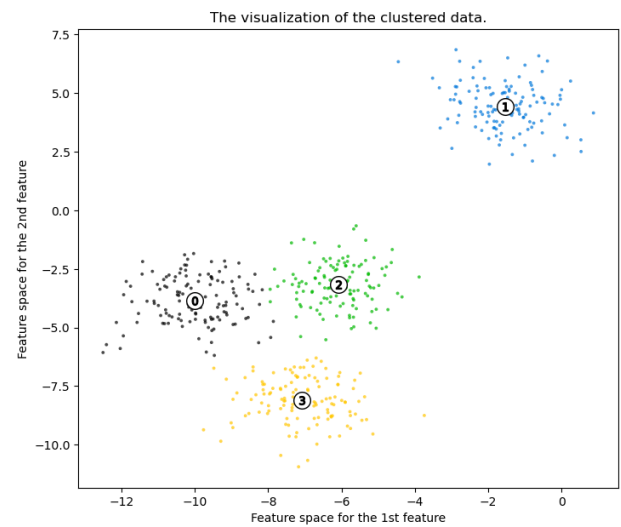
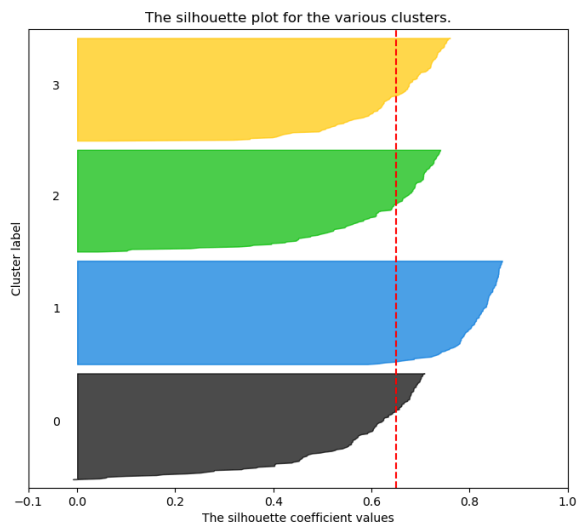
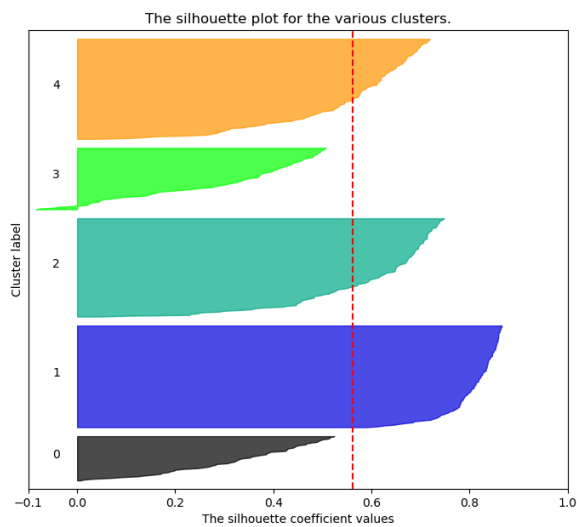
For n_clusters=4, average silhouette score is: 0.6505186632729437

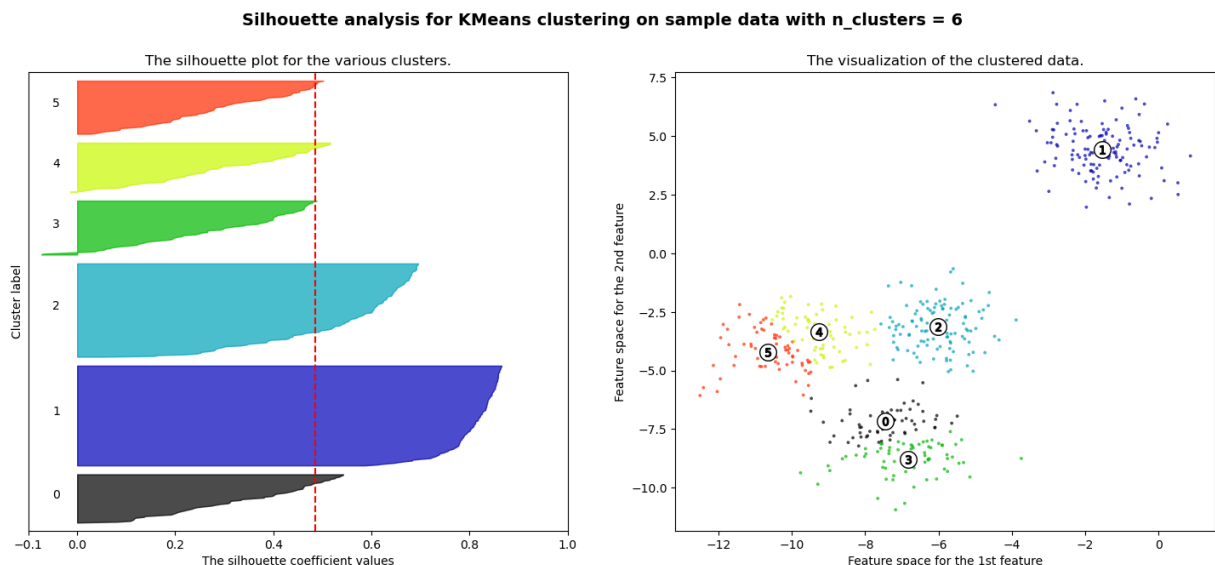
For n_clusters=5, average silhouette score is: 0.561464362648773

For n_clusters=6, average silhouette score is: 0.4857596147013469

Silhouette analysis for KMeans clustering on sample data with n_clusters = 2



Silhouette analysis for KMeans clustering on sample data with $n_clusters = 3$ **Silhouette analysis for KMeans clustering on sample data with $n_clusters = 4$** **Silhouette analysis for KMeans clustering on sample data with $n_clusters = 5$** 



✓ Answer:

Which value of k best suits these data? $K = 2$ best suited this data because it produced the highest average silhouette score (≈ 0.704).

Is that the value you'd expect? no because we saw 4 blobs on the scatter plot so we would expect $k = 4$.

Using what you know about silhouette scores, why is this the case? This happens because silhouette prefers clusters that are very well separated. When we use $K = 2$, the algorithm merges nearby blobs into two larger clusters that are very far apart.

? 3.4 We generated 4 clusters with k-means clustering and 4 clusters with hierarchical clusters from complete linkage. Are these the same clusters? Why or why not? (Consider: Should we expect these methods to match? What is each respective algorithm trying to optimize?)

✓ Answer:

Are these the same clusters? They appear fairly similar, as both methods produce clusters that resemble the original blobs with similar feature values. Additionally, the average silhouette coefficients for both methods are close: approximately 0.65 for K-means (when $K = 4$ and about 0.63 for hierarchical clustering with complete linkage).

However, we should not necessarily expect these methods to produce identical clusters because they optimize different objectives. In K-means clustering, the algorithm tries to minimize the distance between points and their cluster centroid, meaning points within a cluster should be as close as possible to the cluster center. In hierarchical clustering with complete linkage, clusters are formed by minimizing the maximum distance between any pair of points within a cluster.

Because these algorithms optimize different criteria, they do not always produce the same clustering. However, in this dataset the clusters appear similar because the data form four well-separated and compact blobs, making the natural grouping of points clear for both methods.

Midsemester Feedback

Required, 1 pt for completion.

? What is one thing you think these labs have taught well? *Could be a general response, or a more specific comment about a specific lab.*

✓ Answer: Has helped me better understand the algorithms.

? What is one thing you'd like to be different about these labs? *Could be a general response, or a more specific comment about a specific lab.*

✓ Answer: Maybe a bit more guidance on how to implement the algorithms. For example comments describing the steps to be implemented

In []: